34

关于并发的总结对话

教授：那么，你现在头疼了吗？

学生：（吞了两片布洛芬）嗯，有一点。线程（*thread*）交错执行的方式那么多，想起来真费劲。

教授：确实如此。我一直觉得很惊讶：一旦涉及并发执行（*concurrent execution*），短短几行代码就可能变得几乎无法理解。

学生：我也是！身为计算机科学家，居然弄不懂五行代码，说起来有点丢人。

教授：哦，别太难过。如果你翻翻最早那批关于并发算法（*concurrent algorithm*）的论文，会发现它们有时候也是错的！而作者往往还是教授！

学生：（倒吸一口气）教授也会……呃……出错？

教授：是的，确实如此。不过别告诉别人——这是我们的行业机密之一。

学生：我发誓保密。可是，如果并发代码（*concurrent code*）这么难想清楚、这么难写对，我们该怎么写出正确的并发代码呢？

教授：嗯，这才是真正的问题，不是吗？我觉得要从几件简单的事做起。第一，保持简单！避免线程之间复杂的交互；非交互不可的话，就用众所周知、久经考验的方法。

学生：比如简单的加锁（*locking*），也许再加一个生产者/消费者（*producer/consumer*）队列（*queue*）？

教授：没错！这些都是常见的范式，凭你学过的东西，应该能写出可行的解决方案。第二，只在绝对必要时才使用并发（*concurrency*）；能不用就尽量不用。没有什么比过早地优化程序更糟糕的了。

学生：明白了——既然不需要线程，干嘛还要加呢？

教授：正是。第三，如果你确实需要并行（*parallelism*），就去找其他更简单的形式。例如，用 *Map-Reduce* 方法编写并行数据分析代码就是一个绝佳的例子：它实现了并行，却用不着应付锁（*lock*）、条件变量（*condition variable*）以及我们讨论过的其他种种麻烦东西，也就避开了它们那可怕的复杂性。

学生：*Map-Reduce*，是吗？听起来很有意思——我得自己再多读点相关的东西。

教授：很好！你确实应该去读。说到底，你得自己读很多东西，因为我们一起学的内容，相对于外面浩如烟海的知识，只能算最粗浅的入门。读，读，再多读一些！然后动手试一试，写点代码，再多写一些。还要多练习；除了这本书里的内容，外面还有大量其他资源[1]。正如格拉德威尔在他的书《异类》（*Outliers*）中所说，要在一件事上成为真正的专家，大约需要投入一万小时。这可没法全在课堂上完成！

学生：哇，我说不清这是让人泄气还是让人振奋。不过我就当是后者吧，开始干活！该再写些并发代码了……

[1]这里有一个，是游戏的形式：https://deadlockempire.github.io/